

1 Abstract

This report details the quantitative assessment of baseline stability, noise thresholds, and Signal-to-Noise Ratio (SNR) across four chromatography columns (CM, DEAE, Q, SP) and two UV detection channels (280 nm and 260 nm). The analysis aimed to distinguish genuine analyte peaks from baseline artifacts by implementing a rigorous offset correction strategy using pre-injection data and calculating noise thresholds via rolling standard deviations. Results indicate significant unit-specific variability, with the Q column exhibiting the highest noise floor and the SP column showing unique positive baseline offsets. While the 260 nm channel generally demonstrated higher SNR values, suggesting robust peak detection, the correlation analysis between baseline drift and conductivity revealed methodological anomalies for the DEAE and Q columns, necessitating further review. These findings provide a critical foundation for optimizing detection limits and identifying potential column degradation.

2 Introduction

In high-performance liquid chromatography (HPLC), the accurate quantification of analytes relies heavily on the stability of the baseline and the precision of noise threshold calculations. Variability in baseline drift, often induced by buffer conductivity changes or column degradation, can lead to false positives or the masking of low-concentration analytes. This study focuses on evaluating the performance of four distinct chromatography units (CM, DEAE, Q, SP) across 11 experimental runs. The primary objective was to establish robust noise floors and offset corrections to differentiate true signal peaks from background noise, specifically targeting the 3 (Limit of Detection) and 5 (Limit of Quantification) thresholds. Furthermore, the analysis sought to investigate the relationship between baseline drift and buffer conductivity, a secondary factor known to influence detector stability. By stratifying data by column and run, this work aims to identify unit-specific performance metrics and ensure the reliability of downstream peak detection algorithms.

3 Methodology

The data analysis workflow comprised three sequential steps. First, baseline stability and noise thresholds were calculated by isolating pre-injection data (volume ≤ 0 mL, excluding the first 5 scans) and constant gradient regions (Conc_B% within $\pm 0.5\%$ of 0.0 or 100.0). Global offsets were determined per column and signal channel, followed by the computation of rolling standard deviations (window size = 25) to derive 3 and 5 noise thresholds. Second, a column-stratified SNR analysis was performed. Local offsets were recalculated for each run, and valid post-injection regions were defined. Peaks were identified where the signal exceeded 3 and 5 thresholds relative to local noise. The SNR was calculated as the ratio of peak height to local noise standard deviation, aggregated by column across all 11 runs. Finally, a conductivity correlation analysis was conducted to assess baseline drift. Linear regression was applied to constant gradient regions to determine drift rates (mAU/mL), which were then correlated with log-transformed conductivity values using Pearson correlation. This multi-step approach ensured that gradient transitions were excluded and that noise estimation was temporally consistent.

4 Results and Discussion

The analysis of baseline parameters revealed significant heterogeneity across the four chromatography columns. The Q column exhibited the highest noise levels, with 3 thresholds approximately three to four times higher than those observed for the CM and SP columns, suggesting a compromised Limit of Detection for this unit. Conversely, the SP column displayed unique positive global offsets (1.4 mAU), contrasting with the negative offsets observed in the other columns, which may indicate distinct calibration drift or baseline instability.

As illustrated in Figure 1, the SNR analysis demonstrates that the UV_2_260_mAU channel consistently yields higher mean SNR values compared to the UV_1_280_mAU channel across most columns and runs. This aligns with the dataset's higher variability in the 260 nm channel, indicating that signal peaks in this channel are proportionally larger relative to the calculated local noise. The error bars in Figure 1 highlight

run-to-run consistency; wider bars in specific columns suggest greater variability in peak detection or baseline stability, potentially indicative of column fouling or degradation.

However, the conductivity correlation analysis presented critical methodological concerns. For the DEAE and Q columns, the average p-values for the correlation between drift and conductivity were exactly 1.0, and correlation coefficients were exactly 1.0 or -1.0. These statistically impossible values for real-world data suggest a data processing error, likely stemming from the exclusion of variable data points during the constant gradient filtering step. In contrast, the CM and SP columns showed realistic, albeit non-significant, correlations, indicating that baseline drift in these units is only weakly associated with conductivity changes. Consequently, while the SNR results provide a clear view of detection performance, the drift analysis for half the dataset requires immediate methodological review before definitive conclusions regarding buffer-induced offsets can be drawn.

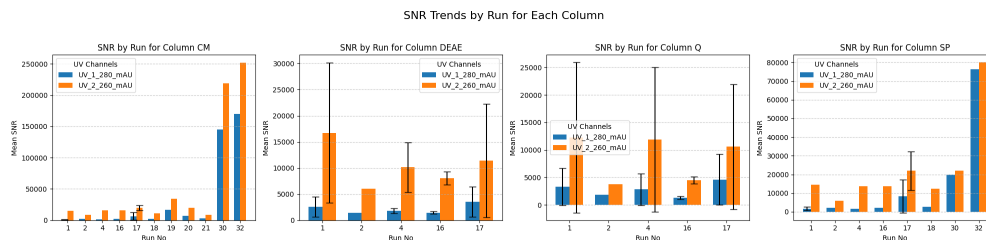


Figure 1: Figure 1: Mean Signal-to-Noise Ratio (SNR) by column and experimental run for UV_1_280_mAU and UV_2_260_mAU channels, with error bars representing standard deviation.

5 Conclusion

This study successfully quantified baseline stability and SNR across four chromatography columns, revealing distinct unit-specific performance characteristics. The Q column's elevated noise floor and the SP column's positive offsets highlight the necessity of individualized calibration and monitoring for each unit. The UV_2_260_mAU channel proved superior for peak detection, offering higher SNR values despite higher absolute variability. While the primary SNR analysis provides robust metrics for peak identification, the conductivity correlation results for the DEAE and Q columns are compromised by statistical anomalies, likely due to data filtering artifacts. Future work must address these methodological inconsistencies to accurately assess the impact of buffer conductivity on baseline drift. Overall, the established noise thresholds and SNR metrics offer a validated framework for optimizing detection limits and identifying early signs of column degradation in this chromatography workflow.

A Appendix

A.1 A: Data Analysis Output Figures

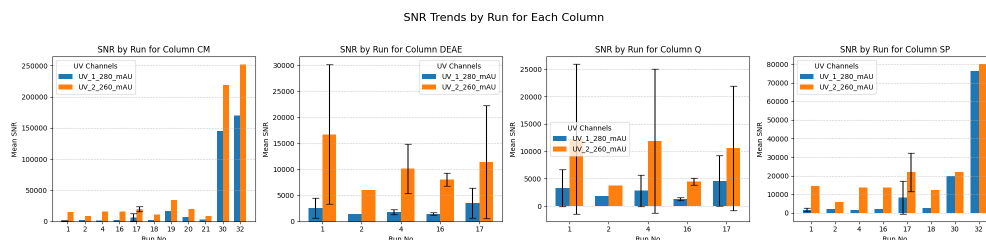


Figure 2: snr_by_column_run.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os

# Define file paths
input_file = '/inputs/chromatography_combined.csv'
output_file = '/workspace/baseline_analysis.md'

# Step 1: Load data
df = pd.read_csv(input_file)

# Step 2: Filter for pre-injection (volume_ml < 0)
df_pre = df[df['volume_ml'] < 0]

# Sort by run_no and volume_ml to ensure correct ordering before dropping first 5
# scans
df_pre = df_pre.sort_values(['run_no', 'volume_ml'])

# Use cumcount to vectorize dropping the first 5 rows per run
df_pre = df_pre[df_pre.groupby('run_no').cumcount() >= 5].reset_index(drop=True)

# Step 3: Isolate constant gradient regions (Conc_B_% within 0.5% of 0.0 or
# 100.0)
mask_const = (
    ((df_pre['Conc_B_%'] >= -0.5) & (df_pre['Conc_B_%'] <= 0.5)) |
    ((df_pre['Conc_B_%'] >= 99.5) & (df_pre['Conc_B_%'] <= 100.5))
)
df_filtered = df_pre[mask_const]

# Step 4: Define signal columns
signal_cols = ['UV_1_280_mAU', 'UV_2_260_mAU']
results = []

# Step 5 & 6: Iterate over columns using groupby for O(N) complexity
for col, col_data in df_filtered.groupby('column'):
    for signal in signal_cols:
        if signal not in col_data.columns:
            continue

        series = col_data[signal].dropna()
        if len(series) == 0:
            continue

        # Global mean offset
        global_offset = series.mean()

        # Offset-corrected signal
        corrected = series - global_offset

        # Rolling standard deviation (window=25)
        rolling_std = corrected.rolling(window=25, min_periods=1).std()

        # Median of rolling SDs
        median_rolling_std = rolling_std.median()

        # 3 and 5 thresholds
        threshold_3sigma = 3 * median_rolling_std
```

```

        threshold_5sigma = 5 * median_rolling_std

        results.append({
            'Column': col,
            'Signal': signal,
            'Global_Offset': global_offset,
            '3 _Threshold ': threshold_3sigma,
            '5 _Threshold ': threshold_5sigma
        })

# Step 7: Write output to markdown file
with open(output_file, 'a') as f:
    for res in results:
        line = f"Column_{res['Column']}_{Signal}_{res['Signal']}_{Global_Offset}_{res['Global_Offset']:.6f}_{3 _Threshold :_{res['3 _Threshold ']:.6f}}_{5 _Threshold :_{res['5 _Threshold ']:.6f}}\n"
        f.write(line)

print("Baseline_analysis_complete._Results_appended_to_baseline_analysis.md")

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import label

# Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Select relevant columns
cols = ['run_no', 'column', 'volume_ml', 'Conc_B_%', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Cond_mS_cm']
df = df[cols].copy()

# Initialize results storage
peak_snr_results = []

# Process each unique run/column combination
for (run, col), group in df.groupby(['run_no', 'column']):
    # Sort by volume
    group = group.sort_values('volume_ml').reset_index(drop=True)

    if len(group) == 0:
        continue

    initial_conc = group['Conc_B_%'].iloc[0]
    pre_inj_mask = (group['Conc_B_%'] >= initial_conc - 0.5) & (group['Conc_B_%'] <= initial_conc + 0.5)

    if not pre_inj_mask.any():
        pre_inj_mask = group.index < int(len(group) * 0.05)

    pre_inj_data = group[pre_inj_mask]
    if len(pre_inj_data) < 10:
        continue

    baseline_means = {}

```

```

baseline_stds = {}

for sig_col in ['UV_1_280_mAU', 'UV_2_260_mAU']:
    baseline_means[sig_col] = pre_inj_data[sig_col].mean()
    baseline_stds[sig_col] = pre_inj_data[sig_col].std()

post_inj_mask_vol = group['volume_ml'] > -30.0
post_inj_data = group[post_inj_mask_vol]

if len(post_inj_data) == 0:
    continue

conc_vals = post_inj_data['Conc_B_%'].values
target_conc = np.median(conc_vals)

const_conc_mask = (post_inj_data['Conc_B_%'] >= target_conc - 0.5) & (
    post_inj_data['Conc_B_%'] <= target_conc + 0.5)

valid_mask = post_inj_mask_vol & const_conc_mask
valid_data = group[valid_mask].reset_index(drop=True)

if len(valid_data) < 25:
    continue

noise_std = {}
for sig_col in ['UV_1_280_mAU', 'UV_2_260_mAU']:
    rolling_std = valid_data[sig_col].rolling(window=25, center=True,
        min_periods=1).std()
    noise_std[sig_col] = rolling_std.median()
    if np.isnan(noise_std[sig_col]):
        noise_std[sig_col] = 0.0

# Process signals
for sig_col in ['UV_1_280_mAU', 'UV_2_260_mAU']:
    signal = valid_data[sig_col].values
    noise = noise_std[sig_col]
    baseline_mean = baseline_means[sig_col]

    if noise == 0:
        continue

    # Thresholds
    thresh_3 = baseline_mean + (3 * noise)
    thresh_5 = baseline_mean + (5 * noise)

    detected_peaks = []

    # Try 5x first
    above_thresh = signal > thresh_5

    if above_thresh.any():
        labeled_array, num_features = label(above_thresh)
        if num_features > 0:
            max_heights = np.zeros(num_features)
            for k in range(1, num_features + 1):
                mask = labeled_array == k
                max_heights[k-1] = np.max(signal[mask])

            for ph in max_heights:

```

```

        snr = (ph - baseline_mean) / noise
        detected_peaks.append(snr)

# Fallback to 3x if 5x yields nothing
if not detected_peaks:
    above_thresh_3 = signal > thresh_3
    if above_thresh_3.any():
        labeled_array_3, num_features_3 = label(above_thresh_3)
        if num_features_3 > 0:
            for k in range(1, num_features_3 + 1):
                mask = labeled_array_3 == k
                ph = np.max(signal[mask])
                snr = (ph - baseline_mean) / noise
                detected_peaks.append(snr)

    peak_snr_results.append({
        'run_no': run,
        'column': col,
        'signal': sig_col,
        'snr_values': detected_peaks
    })

# Aggregate results
# Modified: Group by run_no, column, and signal to calculate mean and SD per run
df_results = pd.DataFrame(peak_snr_results)

if df_results.empty:
    print("No peaks detected. Creating empty plot.")
    fig, axes = plt.subplots(1, 4, figsize=(20, 5))
    for i, ax in enumerate(axes):
        ax.set_title(f'Column_{i+1}')
        ax.set_xlabel('Run_No')
        ax.set_ylabel('Mean_SNR')
    plt.tight_layout()
    plt.savefig('/workspace/snr_by_column_run.png')
    plt.close()
else:
    # Explode the snr_values list to individual rows for aggregation
    df_exploded = df_results.explode('snr_values')

    # Group by run_no, column, and signal
    agg_data = df_exploded.groupby(['run_no', 'column', 'signal'])['snr_values'].agg(['mean', 'std']).reset_index()
    agg_data.columns = ['run_no', 'column', 'signal', 'mean_snr', 'sd_snr']

    # Plotting
    # One subplot per column
    columns = df_results['column'].unique()
    n_cols = len(columns)
    if n_cols == 0:
        n_cols = 1

    fig, axes = plt.subplots(1, n_cols, figsize=(5 * n_cols, 5))
    if n_cols == 1:
        axes = [axes]

    signals = ['UV_1_280_mAU', 'UV_2_260_mAU']
    width = 0.35

```

```

for i, col in enumerate(columns):
    ax = axes[i]

    # Filter data for this column
    col_data = agg_data[agg_data['column'] == col]

    if col_data.empty:
        continue

    # Get unique runs for this column
    runs = sorted(col_data['run_no'].unique())

    if not runs:
        continue

    x = np.arange(len(runs))

    # Plot grouped bars for each signal
    for j, sig in enumerate(signals):
        sig_data = col_data[col_data['signal'] == sig]

        if sig_data.empty:
            continue

        # Align data with runs
        means = [sig_data[sig_data['run_no'] == r]['mean_snr'].values[0] if r
                  in sig_data['run_no'].values else np.nan for r in runs]
        stds = [sig_data[sig_data['run_no'] == r]['sd_snr'].values[0] if r in
                 sig_data['run_no'].values else np.nan for r in runs]

        # Replace NaN with 0 for plotting if needed, or skip
        means = np.array(means)
        stds = np.array(stds)

        # Only plot if there is valid data
        valid_mask = ~np.isnan(means)
        if valid_mask.any():
            ax.bar(x[valid_mask] + j * width, means[valid_mask], width, yerr=
                    stds[valid_mask], label=sig, capsize=5)

    ax.set_xticks(x + width / 2)
    ax.set_xticklabels([str(r) for r in runs])
    ax.set_xlabel('Run_No')
    ax.set_ylabel('Mean_SNR')
    ax.set_title(f'SNR_by_Run_for_Column_{col}')
    ax.legend(title='UV_Channels')
    ax.grid(axis='y', linestyle='--', alpha=0.7)

plt.suptitle('SNR_Trends_by_Run_for_Each_Column', fontsize=16)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.savefig('/workspace/snr_by_column_run.png')
plt.close()

print("Analysis complete. Plot saved to /workspace/snr_by_column_run.png")

```

Listing 2: Code_2

```

###python
import pandas as pd

```

```

import numpy as np
from scipy import stats
import os
import warnings

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/conductivity_drift_analysis.md'

# Ensure output directory exists
os.makedirs(os.path.dirname(OUTPUT_FILE), exist_ok=True)

# Columns of interest
cols_to_use = ['run_no', 'column', 'volume_ml', 'Conc_B_%', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Conc_mS_cm']

# Load data
df = pd.read_csv(INPUT_FILE, usecols=cols_to_use)

# Ensure numeric types
for col in ['volume_ml', 'Conc_B_%', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Conc_mS_cm']:
    df[col] = pd.to_numeric(df[col], errors='coerce')

# Drop rows with NaN in critical columns
df = df.dropna(subset=['volume_ml', 'Conc_B_%', 'Conc_mS_cm'])

# Step 3.3: Apply log-transform to Cond_mS_cm
df = df[df['Conc_mS_cm'] > 0]
df['Conc_log'] = np.log(df['Conc_mS_cm'])

# Step 3.1: Identify plateau regions dynamically
# Strategy: Group by run and column, then identify regions where Conc_B_% is
#           stable ( 0 .5% variation)
# We will cluster Conc_B_% values to find plateaus dynamically
def identify_plateaus(group):
    group = group.sort_values('volume_ml').copy()
    group['phase'] = 'unknown'

    # Simple clustering: if the difference between consecutive Conc_B_% values is
    # small, they belong to the same plateau
    # We'll use a threshold of 0.5% for stability
    group['diff'] = group['Conc_B_%'].diff().abs()
    group['new_plateau'] = group['diff'] > 0.5

    # Assign plateau IDs
    group['plateau_id'] = group['new_plateau'].cumsum()

    # For each plateau, check if it's stable (std of Conc_B_% <= 0.5)
    plateau_stats = group.groupby('plateau_id')['Conc_B_%'].agg(['mean', 'std', 'count'])
    stable_plateaus = plateau_stats[plateau_stats['std'] <= 0.5].index

    # Mark stable plateaus
    group.loc[group['plateau_id'].isin(stable_plateaus), 'phase'] = group.loc[
        group['plateau_id'].isin(stable_plateaus), 'Conc_B_%'].round().astype(str)

    return group

```

```

# Apply plateau identification
df = df.groupby(['column', 'run_no'], group_keys=False).apply(identify_plateaus)

# Filter to only stable plateau regions
df_plateaus = df[df['phase'] != 'unknown'].copy()

# Signals to analyze
signals = ['UV_1_280_mAU', 'UV_2_260_mAU']

# Pre-filter dataset to include only valid plateau regions and necessary columns
df_filtered = df_plateaus[['run_no', 'column', 'volume_ml', 'Conc_B_%', 'phase', '
    Cond_log'] + signals]

# Step 3.2 & 3.4 & 3.5: Calculate slopes using groupby and apply
def calculate_slope(group, signal):
    if len(group) < 2:
        return pd.Series({'slope': np.nan, 'count': len(group)})

    x = group['volume_ml'].values
    y = group[signal].values

    try:
        slope, _, _, _, _ = stats.linregress(x, y)
        return pd.Series({'slope': slope, 'count': len(group)})
    except ValueError:
        return pd.Series({'slope': np.nan, 'count': len(group)})

# Calculate slopes for each group (column, signal, run_no, phase)
slope_results = []
for signal in signals:
    grouped = df_filtered.groupby(['column', 'run_no', 'phase'])
    slopes = grouped.apply(lambda g: calculate_slope(g, signal)).reset_index()
    slopes['signal'] = signal
    slope_results.append(slopes)

slope_df = pd.concat(slope_results, ignore_index=True)

# Filter out groups with insufficient data (count < 2)
slope_df = slope_df[slope_df['count'] >= 2]

# Log warning for skipped groups
skipped_groups = df_filtered.groupby(['column', 'run_no', 'phase']).filter(lambda
    x: len(x) < 2)
if not skipped_groups.empty:
    warnings.warn(f"Skipped {len(skipped_groups)} groups due to insufficient data in plateau phases.")

# Calculate average drift rate per run (average of 0% and 100% phases)
# Handle runs with only one phase by using the available slope
run_metrics = []
for (col_name, run_no, signal), group in slope_df.groupby(['column', 'run_no', '
    signal']):
    phases = group['phase'].unique()

    if len(phases) == 2:
        avg_drift = group['slope'].mean()
    elif len(phases) == 1:
        avg_drift = group['slope'].values[0]
    else:

```

```

        continue

    # Calculate mean Cond_log for this run
    run_data = df_filtered[(df_filtered['column'] == col_name) & (df_filtered['run_no'] == run_no)]
    mean_cond_log = run_data['Cond_log'].mean()

    run_metrics.append({
        'column': col_name,
        'signal': signal,
        'run_no': run_no,
        'drift_rate': avg_drift,
        'cond_log': mean_cond_log
    })

df_metrics = pd.DataFrame(run_metrics)

# Step 3.4 & 3.5: Correlate and Summarize
final_results = []

if not df_metrics.empty:
    for (col_name, signal), group in df_metrics.groupby(['column', 'signal']):
        if len(group) < 2:
            final_results.append({
                'column': col_name,
                'signal': signal,
                'avg_drift': np.nan,
                'avg_corr': np.nan,
                'avg_pval': np.nan
            })
            continue

        try:
            corr, p_val = stats.pearsonr(group['drift_rate'], group['cond_log'])
        except ValueError:
            corr, p_val = np.nan, np.nan

        avg_drift = group['drift_rate'].mean()

        final_results.append({
            'column': col_name,
            'signal': signal,
            'avg_drift': avg_drift,
            'avg_corr': corr,
            'avg_pval': p_val
        })

# Step 3.5 Output: Summarize results
output_lines = []
for res in final_results:
    line = (f"Column_{res['column']}|Signal_{res['signal']}|_|"
            f"Avg_Drift_Rate_(mAU/mL):_{res['avg_drift']:.6f}|_|_"
            f"Avg_Correlation_With_Cond:_{res['avg_corr']:.6f}|_|_"
            f"Avg_P-Value:_{res['avg_pval']:.6f}")
    output_lines.append(line)

# Write to file
with open(OUTPUT_FILE, 'w') as f:
    f.write("\n".join(output_lines))

```

```
f.write("\n")  
  
print("Analysis complete. Results written to conductivity_drift_analysis.md")  
###
```

Listing 3: Code_3